A LECTURE IN SEVEN PARTS

What is Intelligence?

Christoph Kirsch
University of Salzburg, Austria
Czech Technical University in Prague, Czechia

Everything that follows is built from one distinction: the difference between a proof and the truth.

Those two colours keep those two jobs for the whole hour: proof, notation, syntax — truth, meaning, semantics.

FULL DISCLOSURE

This entire talk was generated by an AI.

It took me 40+ years to become someone who could prompt it into existence. I could do that today. I could not have done it yesterday.

I met people on the way vastly more intelligent than me, including some of my students. Without them this talk would not exist.

So, join me in preventing our smartphones, social media, and AI from distracting us, and help such people find you by becoming who you are and no one else. This talk is about that.

ACKNOWLEDGEMENT

This is a collaborative effort.

Leoni Brand
Eva Jonas
Julius Möller
Stefan Reiß

Department of Psychology
University of Salzburg, Austria

They inspired me, encouraged me, and helped me leave my computer-science comfort zone — and reach out to everyone affected by AI.

NOT BY REPEATING

The booming. The dooming. Both are forecasts about a product cycle, and both expire on the next one.

BUT BY ACADEMIC GROUNDING

Results that were already true before this technology arrived, and will still be true after whatever replaces it. That is the whole of what follows.

THE QUESTION · AND WHY WE WON'T ANSWER IT

"Is it intelligent?" is a badly posed question.

It asks about a hidden property of a system, in a language — English — with no precise semantics for the word *intelligent*.

Psychology met this first and answered it best: you cannot measure a construct, only an operational definition of it — and the definition is never the thing. A century of psychometrics is a century of taking that gap seriously.

We will spend the next hour earning the right to ask a better question, one that stays valid no matter what technology arrives next.

> *The question of whether machines can think is about as relevant as the question of whether submarines can swim.*
>
> EDSGER W. DIJKSTRA, 1984

THE SHORT ANSWER · STATED NOW, EARNED LATER

Intelligence is finding the next language — which means finding truth you cannot yet prove.

1. Intelligence develops new formal languages — or at least new *properties*, expressible and provable in languages we already have.
2. That requires finding and understanding promising unproven truth. You apprehend something as true while no proof exists; the notation is built afterwards, to hold what you already saw.
3. Nothing hands you that step. No rule, no procedure, no quantity of compute derives it — a result we will *prove*, not assert. So it is a challenge for a first-year student, for a Nobel laureate, and for any AI, equally.

The rest of the hour is the derivation — and the reason this is good news for everyone in this room, whatever you are studying.

ROUTE

Seven parts.

I	Vastness	why 34 bytes beat the universe
II	Infinity	why meanings outnumber notations
III	Self-reference	why proof ≠ truth
IV	Cost	time, space, energy
V	Everywhere	biology, mind, money, music
VI	Machines	what today's AI cannot escape
VII	Practice	a method with no expiry date

Four of these parts end in a theorem that sounds like bad news. All four are good news. That reversal is the point of the lecture.

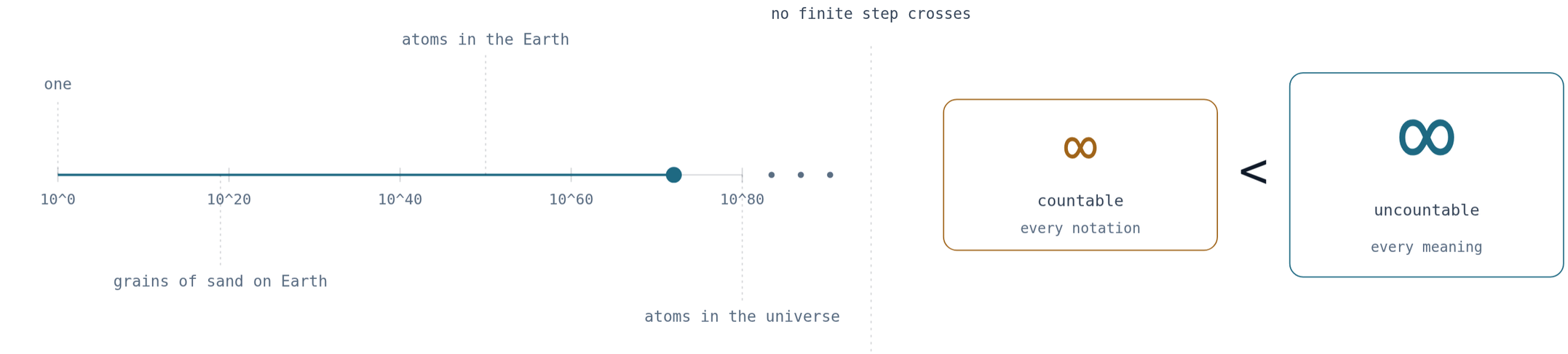
PART I

Vastness

First a feeling for size — then the smallest possible distinction, and everything it builds.

THE MAP

One line carries the whole talk: small, vast, then two sizes of endless.



Left. From one thing to every atom there is — the whole physical world in eighty steps, each one ten times the last.

The wall. No number of steps crosses it. Infinity is not the far end of the line; it is what the line never reaches.

Right. Past the wall, endlessness comes in sizes: one counts every notation, the other every meaning — and is strictly bigger.

ANCHORS

A million, a billion, a trillion. Three words one syllable apart — three different worlds.

10^6 s

A MILLION SECONDS · 11½ DAYS

10^9 s

A BILLION SECONDS · 32 YEARS

10^{12} s

A TRILLION SECONDS · 32,000 YEARS

A million seconds ago you had not yet packed for today. A billion seconds ago you were a child, or not yet born. A trillion seconds ago somebody was painting the walls of the Chauvet cave.

Each step is three zeros — the same three zeros, three times over. The words rhyme, so the mind files them together and treats the gaps as small. The gaps are not small. They are the subject.

THE HABIT

Never accept a number you cannot picture. Find the anchor — a fortnight, a career, an ice age — or admit you are repeating the number, not reading it.

UNITS

Kilo, mega, giga, tera: the same three zeros, again and again.

PREFIX	FACTOR	BYTES · HOW MUCH IT HOLDS	HERTZ · HOW OFTEN IT HAPPENS
kilo	10^3	KB — a long paragraph	kHz — the top of human hearing, 20 kHz
mega	10^6	MB — a novel	MHz — the first IBM PC, 4.77 MHz, 1981
giga	10^9	GB — a thousand novels	GHz — this laptop; Wi-Fi at 2.4
tera	10^{12}	TB — a million novels	THz — infrared light

A byte is eight bits — one keystroke. A hertz is one beat per second. The left column counts things, the right counts events, and the prefixes do not care which.

HOLD A NANOSECOND

At 1 GHz a beat lasts a billionth of a second, and light gets 30 cm in that time. Grace Hopper handed those 30 cm out as lengths of wire, so a room could feel why a signal cannot cross a desk and return inside one beat.

TWO KILOS

Kilo means a thousand. In memory it means 1,024. Both are in daily use.

	BASE 10	BASE 2	GAP
kilo	KB = 10^3	KiB = 2^{10} = 1,024	+2.4%
mega	MB = 10^6	MiB = 2^{20}	+4.9%
giga	GB = 10^9	GiB = 2^{30}	+7.4%
tera	TB = 10^{12}	TiB = 2^{40}	+10.0%

Memory is addressed with bits, so its natural steps are powers of two — and 2^{10} lands so close to 1,000 that the prefix was simply borrowed. Every further rung widens the gap.

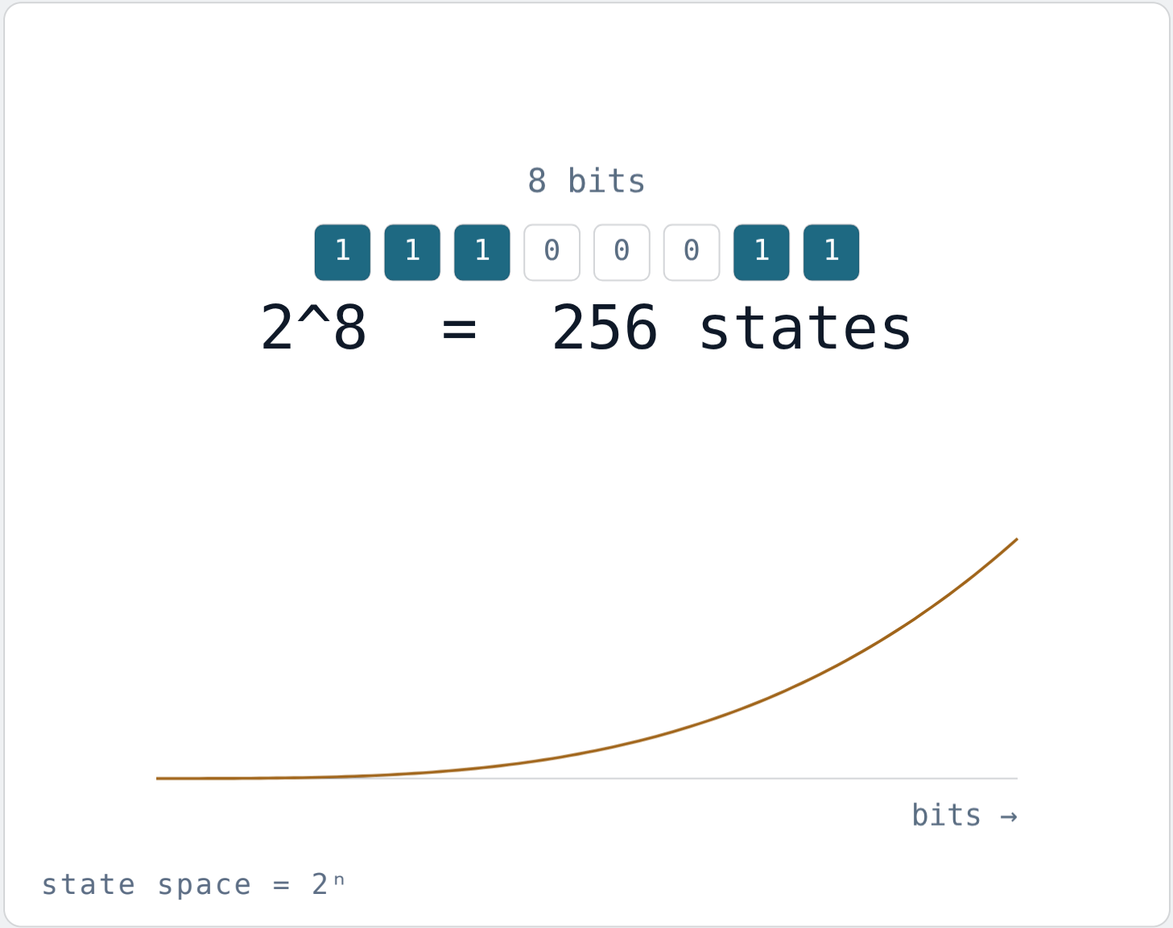
Which one you were handed depends on the trade. Memory comes in powers of two; disks and networks in powers of ten — a "1 TB" drive holds 10^{12} bytes and shows up as 931 GiB. Hertz is never base two, and neither is the kilo in kilometre.

NOTICE WHAT JUST HAPPENED

One notation, two meanings, and nothing in the notation to tell you which you were handed. Keep hold of that. The rest of the lecture is about that gap.

ONE BIT

A bit is one distinction: this, not that.



Add a bit and you don't *add* states. You double them.

Ten bits: a thousand states. Twenty: a million. Thirty: a billion.

Doubling is the most underestimated operation in human reasoning — and the entire engine of computing.

SCALE CHECK

34 bytes of memory has more states than the observable universe has atoms.

10^{80}

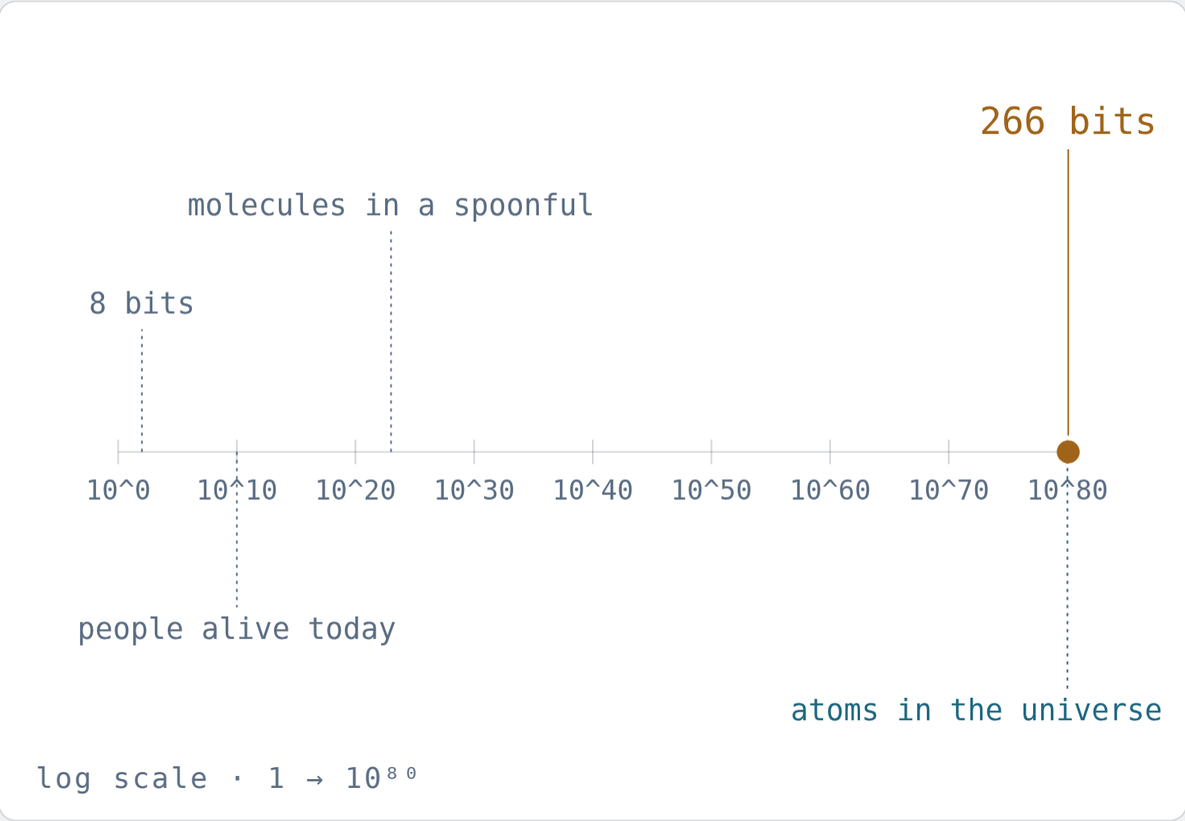
ATOMS IN THE OBSERVABLE UNIVERSE

2^{266}

STATES OF 266 BITS ≈ 1.2×10^{80}

37

BYTES TO BEAT EVERY PHOTON, ~10^{89}



YOUR POCKET

A phone with 8 GB of memory has $2^{68{,}719{,}476{,}736}$ states.

Written out in decimal, that number has about 20.7 billion digits.

At 3,000 digits a page: 6.9 million pages. Bound into 500-page books: 13,800 volumes — some 400 metres of shelf.

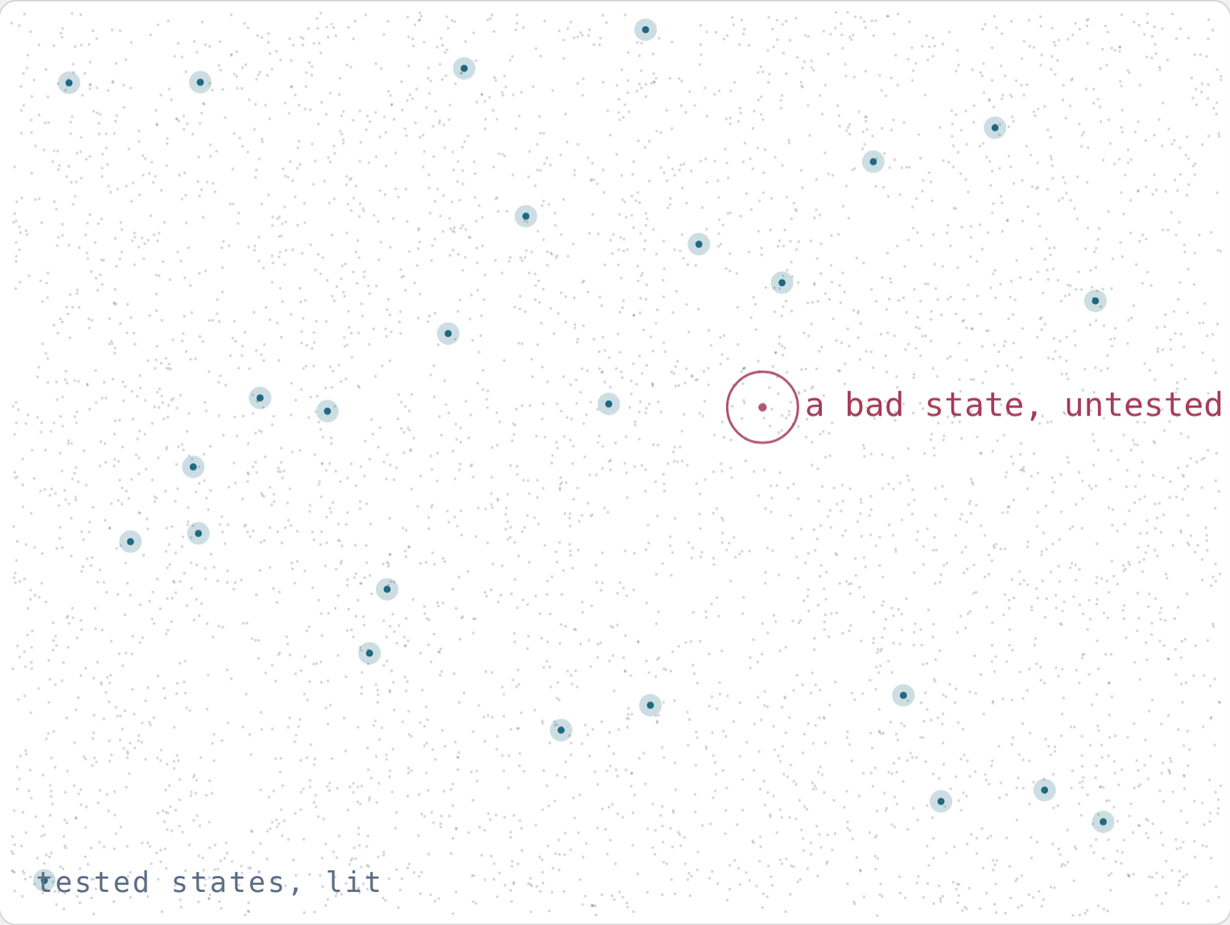
That shelf does not hold the phone's states. It holds *the number* of them, written down once.

```
2^68719476736 = 0436191480101874033536267652159723289730354775350314382935925663596540511366864144104064630223682222771703373641373784899092270222012023998613352763825008185532638120468754732386765121435789453773124001636639369349296569407095267716214536743608782792262942691661830453232205324959738133215497510205751571746789228898244907141844290131327216
```

Every app you will ever install, every photo you will ever take, and every bug you will ever hit is one point in this space.

THE CONSEQUENCE

In a space that big, good states and bad states look alike.



Test a billion states per second, starting at the Big Bang, and by now you would have checked about 2^{88} of them.

Out of $2^{68,719,476,736}$. The fraction is not small. It is indistinguishable from zero.

> *Testing shows the presence, not the absence of bugs.*
>
> EDSGER W. DIJKSTRA, 1969

REFRAME · 1 OF 5

The vastness is not the problem. It is the inventory.

WHAT IT FORBIDS

Certainty by inspection. You will never check your system exhaustively — not with more testers, not with faster machines, not ever.

WHAT IT OPENS

Every symphony not yet written, every protein not yet folded, every proof not yet found, every program not yet imagined — all of them are already in there, waiting to be addressed.

The same enormity that hides your bugs is the reason novelty is inexhaustible. You cannot have one without the other.

INTERLUDE · LITERATURE

Borges built this space in 1941 and called it a library.

The Library of Babel holds every possible book: every truth, every refutation — and overwhelmingly, gibberish.

Its inhabitants go mad: containing every truth is worthless without a way of finding one.

Modern echo: a large language model already "contains" astonishing amounts of text. Containment was never the hard part.

BORGES · 1941

410 pages, 40 lines, 80 characters, 25 symbols. The shelves hold $25^{1{,}312{,}000}$ books.

$\approx 10^{1{,}834{,}097}$ volumes

THE SHIFT

From *storage* to *search*: from having the answer somewhere to knowing it when you see it.

PART II

Infinity

Vast is still finite. Now we count things that aren't — and discover that some infinities are bigger than others.

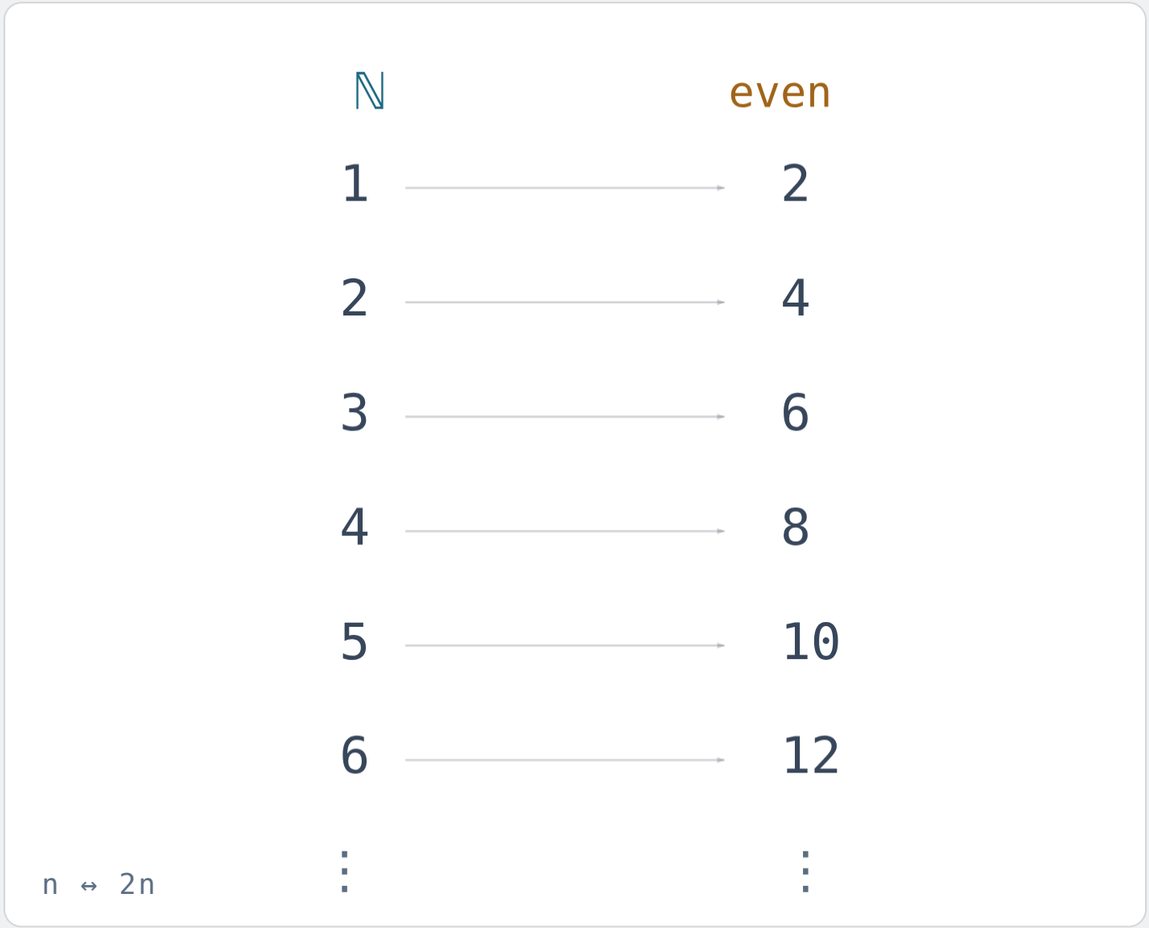
COUNTING WITHOUT NUMBERS

Two sets are the same size if you can pair them up.

No counting needed — just a perfect pairing. Every left has exactly one right, and nothing is left over.

So there are as many even numbers as numbers: pair n with 2n. The part is as big as the whole.

A set pairable with 1, 2, 3, … is called countable. It can be listed.



NOTATION

Everything we can write down is countable.

A program is a finite string of symbols. So is a proof, a specification, a sentence, a score, a formula, a prompt.

List all strings of length 1, then length 2, then 3 … every finite text appears at some finite position.

So: all programs that will ever exist form a countable list. Same for all proofs. Same for all sentences of English.

1. ε
2. 0
3. 1
4. 00
5. 01
6. 10
7. 11
8. 000
9. 001
10. …

every finite text has a number

the enumeration of all texts

THIS HAS A NAME

Assigning a number to each piece of syntax is *Gödelisierung* — Gödel numbering. It is how a machine reads a machine, and we return to it in Part III.

MEANING

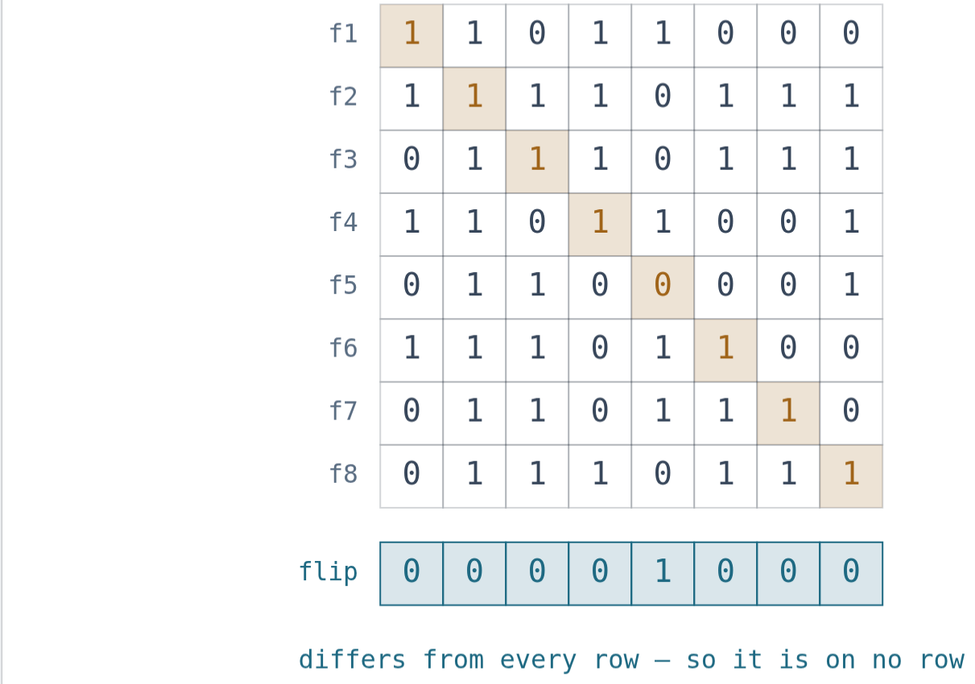
What we want to talk about is not.

Consider all infinite sequences of bits: `0110100011…` forever.

Each one is a complete answer to an endless list of yes/no questions — a behaviour, a function, a real number, a fate.

Claim: no list can contain them all. Not a long list. Not an infinite list. No list.

Cantor's proof is three lines and it is the most consequential argument of the 20th century.



THE OBJECTS

A subset of the numbers is one yes-or-no answer per number.

Take any collection of counting numbers — the evens, the primes, just {3}, none of them, all of them. Each one is a subset.

To pin one down, walk 1, 2, 3, … and answer *in* or *out*, forever. That answer sheet is an infinite bit string, and every infinite bit string is one.

All of them together make the power set, written $2^{\mathbb{N}}$ — two choices, made $\mathbb{N}$ times.

A handful have names. Almost all are an endless coin-flip, with nothing shorter to say about them than the flips.

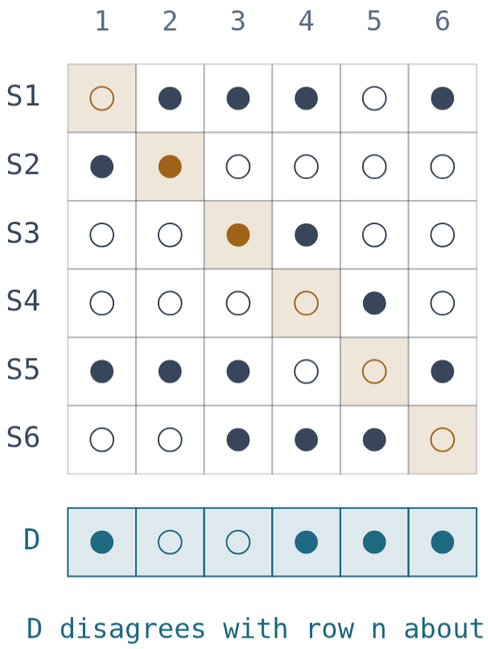
	1	2	3	4	5	6	7	8	…
{ }	0	0	0	0	0	0	0	0	…
all	1	1	1	1	1	1	1	1	…
evens	0	1	0	1	0	1	0	1	…
primes	0	1	1	0	1	0	1	0	…
{3}	0	0	1	0	0	0	0	0	…
no name	1	1	0	1	0	1	1	0	…

and almost every subset is this one

subsets as answer sheets · 1 = in

THE PROOF · 1 OF 2 · SUBSETS

Hand me a list of every subset. I will build one that is not on it.



Suppose the subsets could be listed: `S1, S2, S3, …`, every single one of them somewhere on the list.

Go down the diagonal and ask each row about *its own number*. Is 1 in `S1`? Is 2 in `S2`? Is `n` in `Sn`?

Now build `D` by answering the opposite every time: `D = { n : n is not in Sn }`. Nothing exotic — a rule anyone can apply, one number at a time.

`D` is not `S1`: they disagree about 1. Not `S2`: they disagree about 2. Not `Sn` for any `n` whatsoever. So the list left something out — and it was any list at all.

THE PROOF · 2 OF 2 · NUMBERS

The same move, on the numbers between 0 and 1.

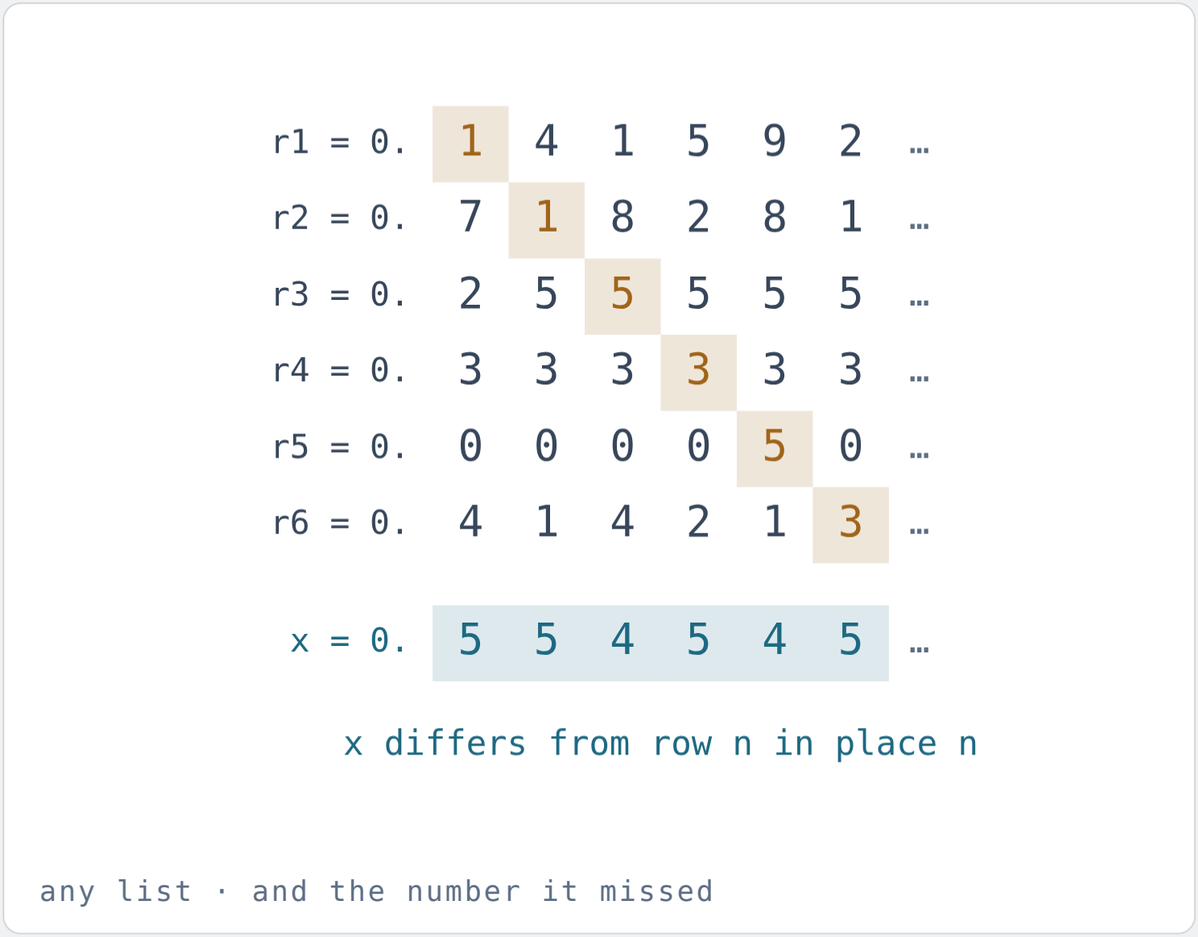
Suppose you could list them: r1, r2, r3, …, each written out as an endless decimal.

Take the first digit of the first, the second digit of the second, the n-th of the n-th. The diagonal again.

Build a new number x by changing every one of those digits. Then x differs from r1 in the first place, from r2 in the second, from rn in the n-th — so x is on no row, and the list was not a list of all of them.

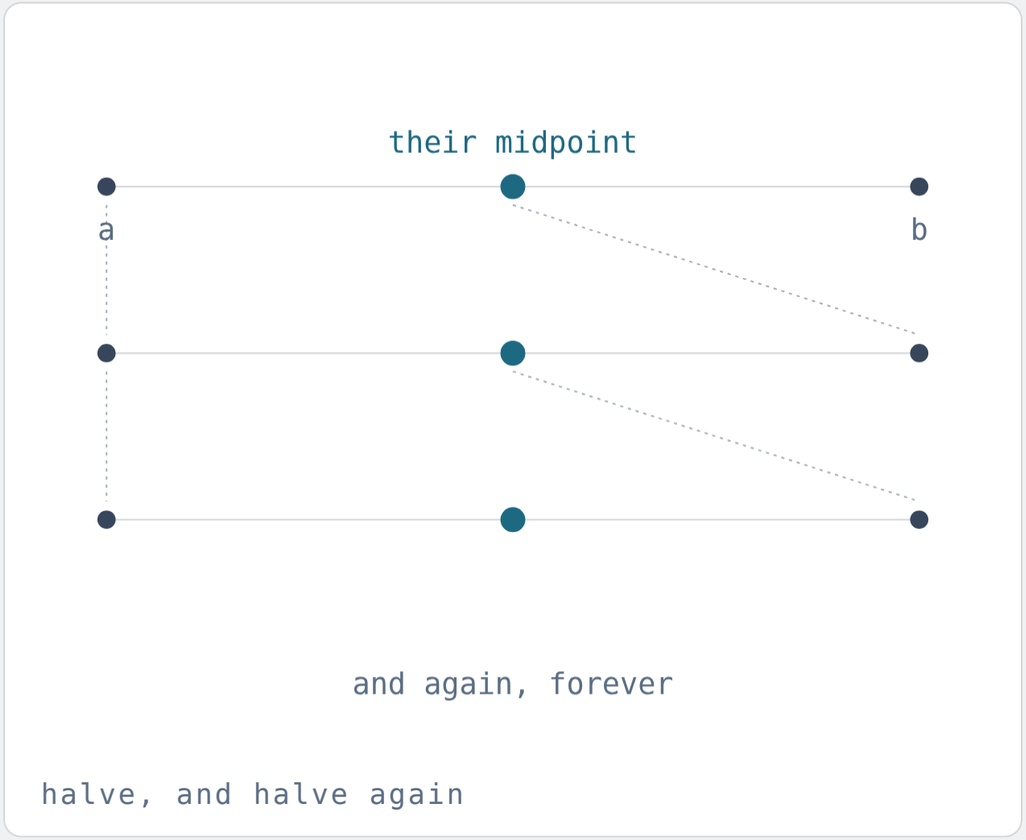
THE ONE PLACE IT NEEDS CARE

Change each digit to 5, or to 4 if it was already 5. That keeps you clear of the trailing nines: 0.4999… and 0.5000… are the same number written twice, and a proof that only landed on *that* would have proved nothing.



INTUITION

Between any two numbers, however close, there is another.



Pick two reals as close together as you like. Their midpoint lies strictly between them. Now take those two — and do it again, forever.

There is no *next* real number: nothing to step from and nothing to step to. A list is nothing but firsts and nexts, which is the wrong shape for a line with no grain.

BUT DO NOT LET IT DO THE WORK

Density is the feeling, not the reason. The *fractions* are dense in the same way — between any two there is another — and they can still be listed, by walking a grid of numerators and denominators corner by corner. Only the diagonal tells the two cases apart.

THEOREM

The diagonal, once and for all.

> CANTOR · 1891
>
> *For every set S, the set of all subsets of S is strictly larger than S. In particular, the real numbers cannot be listed.*

WHERE SELF-REFERENCE ENTERS

The proof builds an object *from* the list that asks of each row: "do you contain yourself here?" — and then answers the opposite. The list is used against itself.

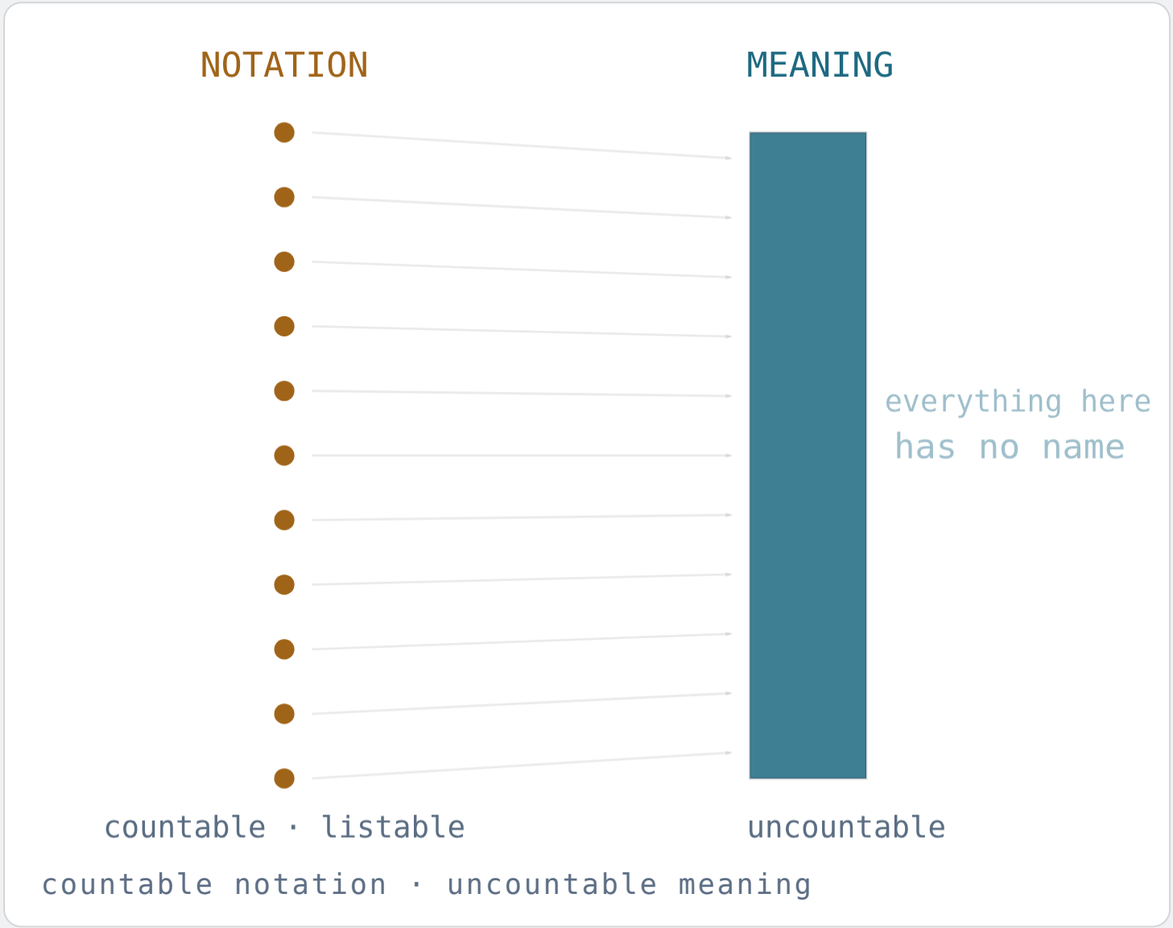
YOU HAVE JUST SEEN IT TWICE

Once on subsets, once on decimals — and it was the same proof both times, because a subset *is* an infinite bit string and so is a decimal expansion. Power set, real numbers, bit sequences: three costumes, one cardinality. The theorem says it for every S at once.

Russell would turn the same trick on set theory itself in 1901: *the set of all sets that do not contain themselves*. Same diagonal, aimed at the foundations.

THE RESULT EVERYTHING HANGS ON

There are incomparably more meanings than notations.



Programs: countable. Behaviours: uncountable. So almost every behaviour has no program.

Proofs: countable. Truths about numbers: uncountable. So almost every truth has no proof.

Sentences: countable. Distinctions the world admits: uncountable. So almost everything is unsaid.

"Almost every" here is exact: the expressible is a vanishing sliver, of measure zero, inside the meaningful.

REFRAME · 2 OF 5

Scarcity of notation is a job description.

WHAT IT FORBIDS

A final language. No vocabulary — mathematical, legal, musical, or neural — will ever cover the space of meanings.

WHAT IT OPENS

An inexhaustible supply of things worth naming. Every new notation captures meaning that was previously unreachable — and there is always more left.

Calculus, double-entry bookkeeping, staff notation, the periodic table, chemical formulae, DNA sequencing, type systems: each was a raid on the uncountable. The supply of raids never runs out.

PART III

Self-Reference

Counting told us the gap exists. Self-reference walks us to the edge of it and points.

SYNTAX AND SEMANTICS

The score is not the music.

SYNTAX · NOTATION

Marks on paper. Finite, discrete, checkable, copyable. `1 + 1`, a staff of crotchets, H_2O, a line of code, this sentence.

SEMANTICS · MEANING

What the marks are *about*. The number two. Sound in a room. A molecule that dissolves salt. A machine's behaviour over all inputs.

THE MAP

A semantics is a *function* from notation to meaning. Defining that function precisely is the founding act of every exact discipline.

Frege 1892 · Sinn / Bedeutung

Saussure · signifier / signified

Korzybski 1931 · the map is not the territory

Magritte 1929 · ceci n'est pas une pipe

Peirce · sign / object / interpretant

TWO KINDS OF LANGUAGE

Natural language is compression. Formal language is commitment.

"I saw the man with the telescope." Two readings, one string. English resolves it with a shared world you and I already have.

That shared world is why English is powerful — and why it cannot be a foundation. Its semantics is us, and we differ.

A formal language pays a price — narrowness, pedantry, effort — to buy one thing: a meaning that does not depend on who is reading.

NATURAL	FORMAL
ambiguous	single-valued
elastic, forgiving	brittle, exact
persuades	proves
needs a mind	needs a machine
learned by living	defined by decree

Prompting an AI in English is negotiation. Writing a test, a type, a schema, a unit, a contract is legislation.

GÖDELISIERUNG

Give every text a number, and notation becomes something you can compute with.

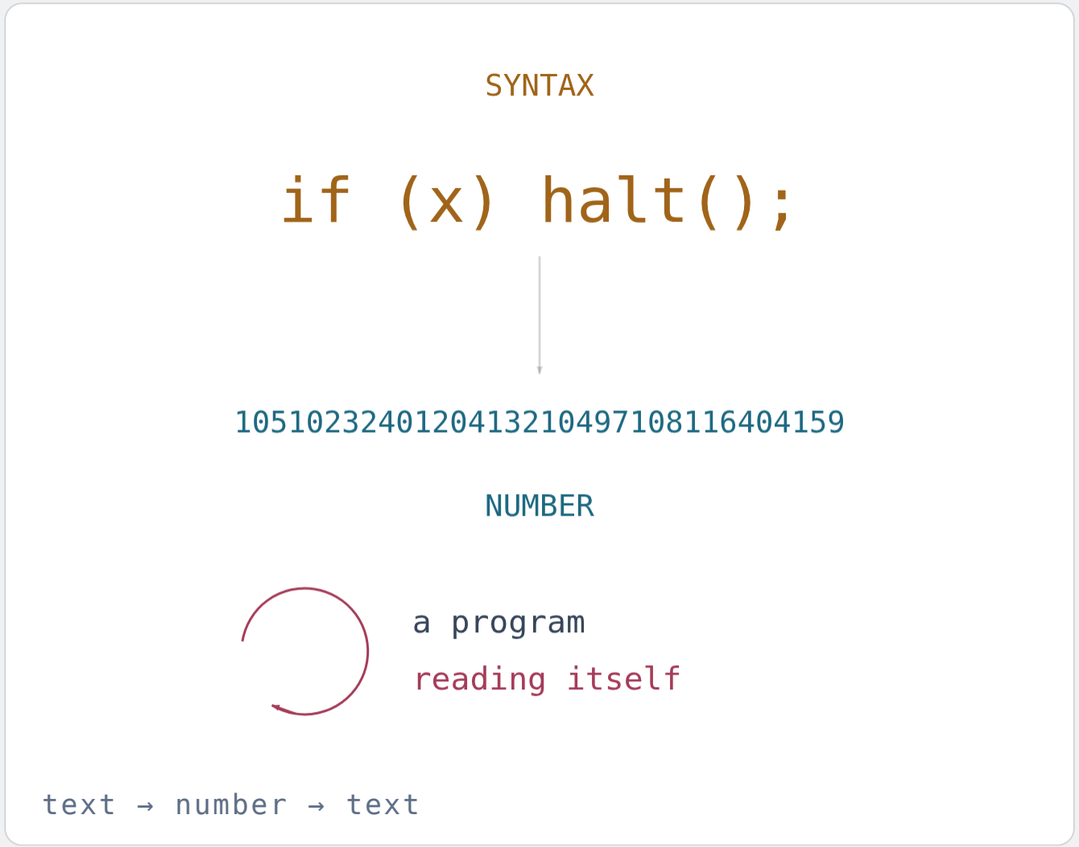
Gödel's device, 1931: encode formulas, proofs and programs as integers. Now arithmetic can talk about arithmetic.

Every computer you have ever used is this idea in metal: code is data. A compiler eats programs. An operating system runs programs. A model is trained on programs.

And once a system can describe systems, it can describe itself. Self-reference is not a trick; it is the price of being expressive enough to be useful.

ASIDE · FOR THE COMPUTER SCIENTISTS

In *selfie*, a 12KLOC C* system, a compiler compiles its own source and an emulator executes that code — including itself. Gödelisierung you can run in a terminal.



THEOREM

There are truths with no proof.

GÖDEL · FIRST INCOMPLETENESS · 1931

> *Any consistent formal system rich enough to describe arithmetic contains statements that are true but not provable within it.*

HOW

Build, by Gödelisierung, a sentence that says: *"this sentence has no proof in this system."* If it were provable, the system proves a falsehood. So it is unprovable — and therefore true.

THE SAME DIAGONAL

Cantor built a row not on the list. Gödel builds a truth not on the list of provable things. Adding it as a new axiom does not help: the construction simply runs again.

Proof is a finite object you can check. Truth is not.

Proof is syntax. Truth is semantics. They are not the same size.

THEOREM

And no strong system can certify itself.

> GÖDEL · SECOND INCOMPLETENESS · 1931
>
> *Such a system cannot prove its own consistency — unless it is inconsistent, in which case it proves everything.*

READ IT AS ENGINEERING

A trustworthy system cannot be the source of its own trust. Confidence has to come from *outside*: a stronger theory, an experiment, an independent auditor, reality.

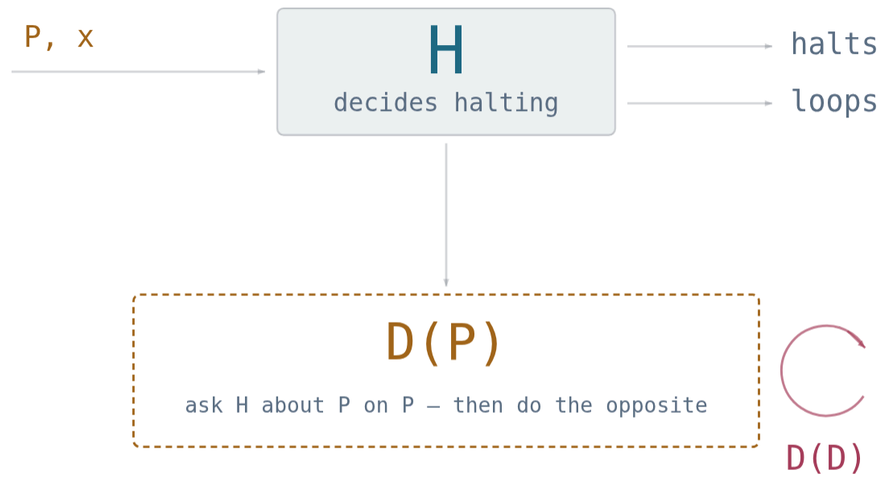
READ IT AS ADVICE

When a model explains why its own answer is correct, you have received more output from the same system — fluent, plausible, and not a certificate. Introspection is not audit.

Tarski, 1936, closes the circle: no sufficiently expressive language can define *truth* for its own sentences. Truth always lives one level up.

UNDECIDABILITY

Will this program ever stop? No program can always tell.



halts ⟺ ¬halts — so H cannot exist

feed the decider its own description

Suppose `H(P,x)` decides, for every program and input, whether `P` halts on `x`.

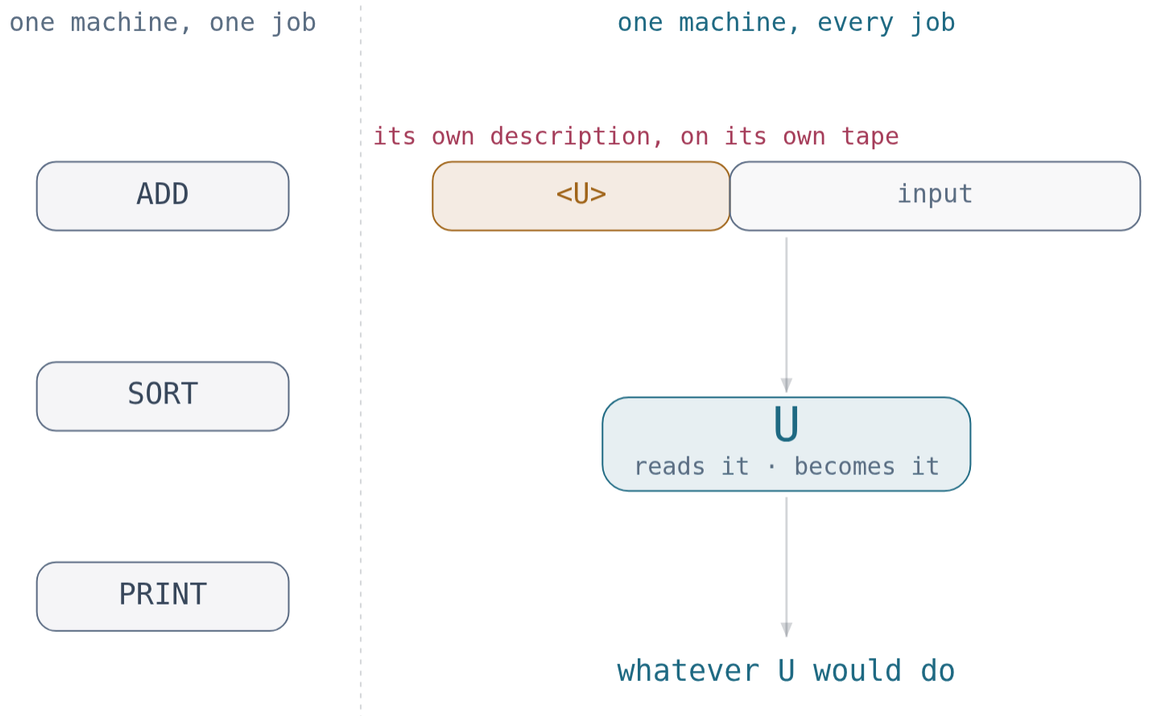
Build `D(P)`: ask `H` whether `P` halts on `P` — then do the opposite.

Now run `D(D)`. It halts exactly if it doesn't. Contradiction. So `H` never existed.

Turing, 1936 — the same paper that defined the universal machine, and thus invented the computer. The limit and the machine arrived together.

THE SAME MOVE, FORWARDS

One machine that can be any machine.



the job moved from the wiring to the tape

A machine used to *be* its job — to sort instead of add you built a different machine. Turing's move: put the machine's description *on the tape*, as data.

Then one machine U reads any description and does whatever that machine would do. That is universality: one piece of hardware, every possible behaviour, because the behaviour arrives as notation.

Your phone is not phone-shaped. It is U holding a description — which is why a new app needs no new hardware.

ONE SENTENCE, READ TWICE

Hand a decider its own description: contradiction, no H. Hand a machine *any* description: every program at once, one U. Turing published both in 1936.

THE GENERAL CASE

It is not just halting. It is every interesting question about meaning.

> RICE · 1953
>
> *Every non-trivial property of the behaviour of programs is undecidable. Only questions about their text are safe.*

DECIDABLE · ABOUT SYNTAX

How long is the code? Does it parse? Does it use this library? Are the types consistent? Does it contain a loop?

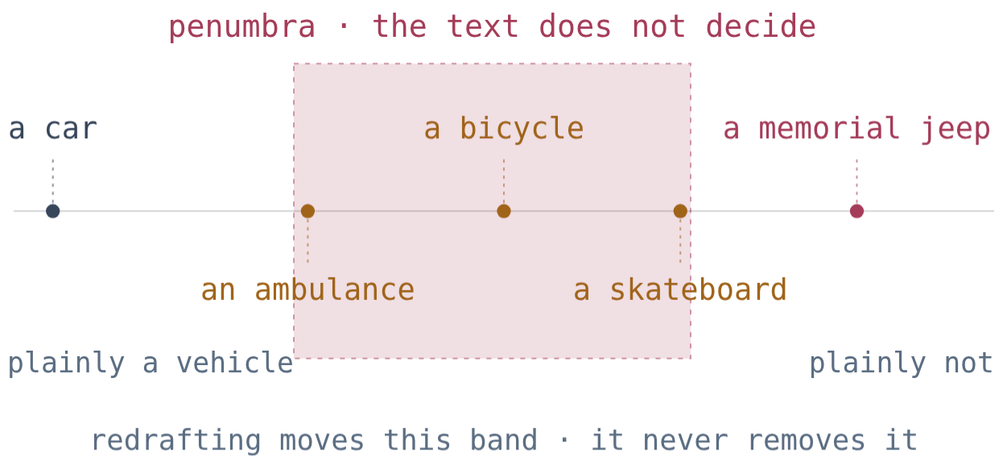
UNDECIDABLE · ABOUT SEMANTICS

Is it correct? Is it equivalent to that one? Does it ever leak the key? Is it free of infinite loops? Is it safe?

So every practical tool — a type checker, a test suite, a linter, a model checker, a fuzzer, a proof assistant — is a deliberate approximation: sound but incomplete, or complete but unsound, or exact only within a bound. Choosing which to give up is the discipline.

INTERLUDE · LAW

No text contains its own application.



Hart's core, and Hart's penumbra

A statute says `no vehicles in the park`. A car, plainly. And then the arguing starts.

Hart called the easy cases the core, the arguable ones the penumbra. No redrafting removes it — only moves it. The words are finite; the situations are not.

So law stops trying to settle meaning in the text and builds an institution instead: courts, appeals, precedent. Not a workaround — the only available design.

GÖDEL'S OTHER THEOREM

At his 1947 citizenship hearing, Einstein beside him, Gödel announced a self-referential flaw by which the Constitution could be legally turned into a dictatorship. The judge steered him off it, and nobody recorded which flaw he meant.

PATTERN

One idea. Sixty years. Six theorems.

YEAR	WHO	THE LIST	THE DIAGONAL OBJECT
1891	Cantor	all real numbers	a number on no row
1901	Russell	all sets	the set of non-self-members
1931	Gödel	all provable sentences	"I am unprovable"
1936	Tarski	all definable predicates	"I am false"
1936	Turing	all decidable questions	a program that defies its judge
1953	Rice	all semantic properties	all of the above, at once

Assume a complete list. Ask each item about *itself*. Answer the opposite.

Learn the move once and you own the century.

REFRAME · 3 OF 5

No final authority means no ceiling.

WHAT IT FORBIDS

A machine, a method, or a person that settles all questions. No complete rulebook, no self-certifying system, no automatic correctness.

WHAT IT OPENS

Mathematics is not a finished building but an open frontier; engineering is a craft rather than a lookup; and judgement — yours — never becomes redundant.

> *Beware of bugs in the above code; I have only proved it correct, not tried it.*
>
> DONALD E. KNUTH, 1977

PART IV

Cost

Suppose a question *is* decidable. You still have to pay for the answer — in time, space, and energy.

DECIDABLE ≠ DOABLE

A question with a guaranteed answer you will never receive.

Given a logical formula over 100 yes/no variables: is there an assignment making it true? Perfectly decidable — try all 2^{100}.

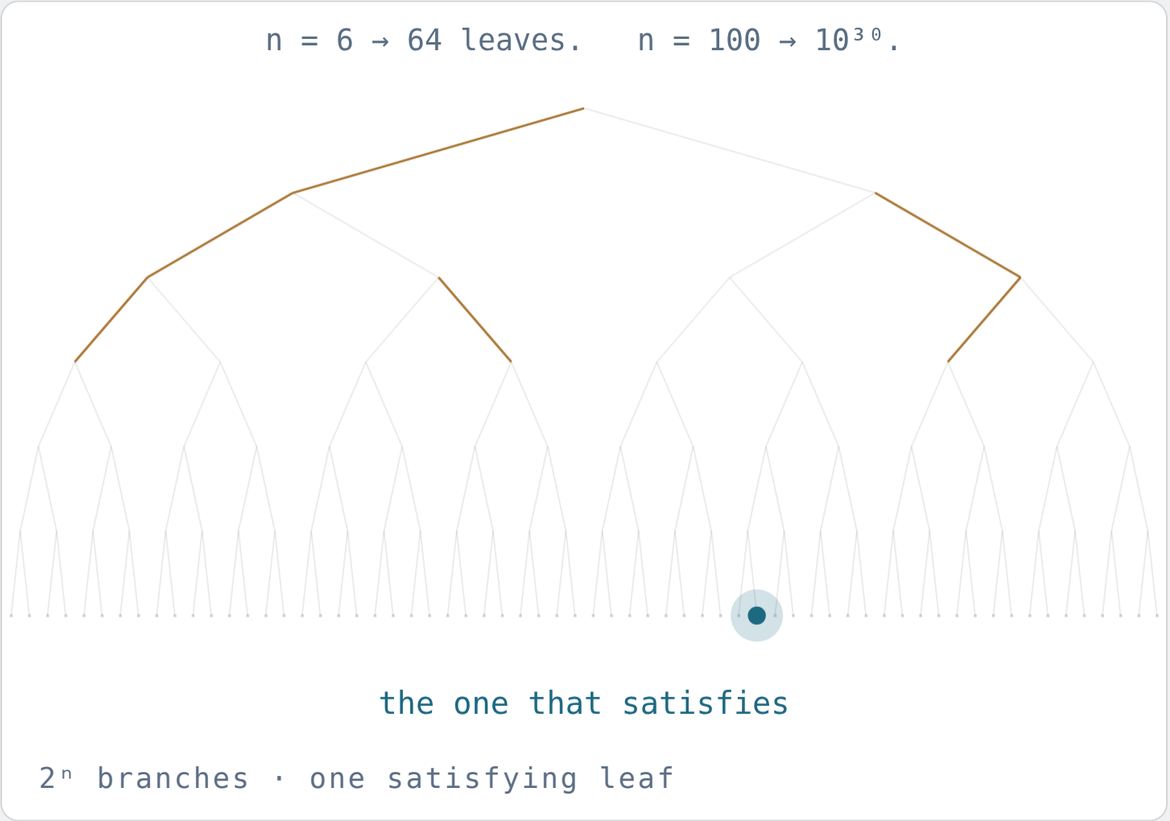
2^{100}

≈ 1.3×10^{30} ASSIGNMENTS

10^{13} years

AT A BILLION CHECKS PER SECOND

Chess has ~10^{44} legal positions; Go, ~10^{170}. Nobody "solves" these. We navigate them — with heuristics, structure, and luck.



INTRACTABILITY

Hard to find. Easy to check.

That asymmetry has a name: NP — problems whose solutions are quick to verify even if finding them seems to need exponential search.

Cook and Levin, 1971–73: thousands of such problems are *the same problem* in disguise. Crack one efficiently and you crack scheduling, routing, folding, packing, proving.

Whether that is possible — `P = NP?` — is the most consequential open question in the exact sciences. Most researchers bet no.

THE ASYMMETRY, EVERYWHERE

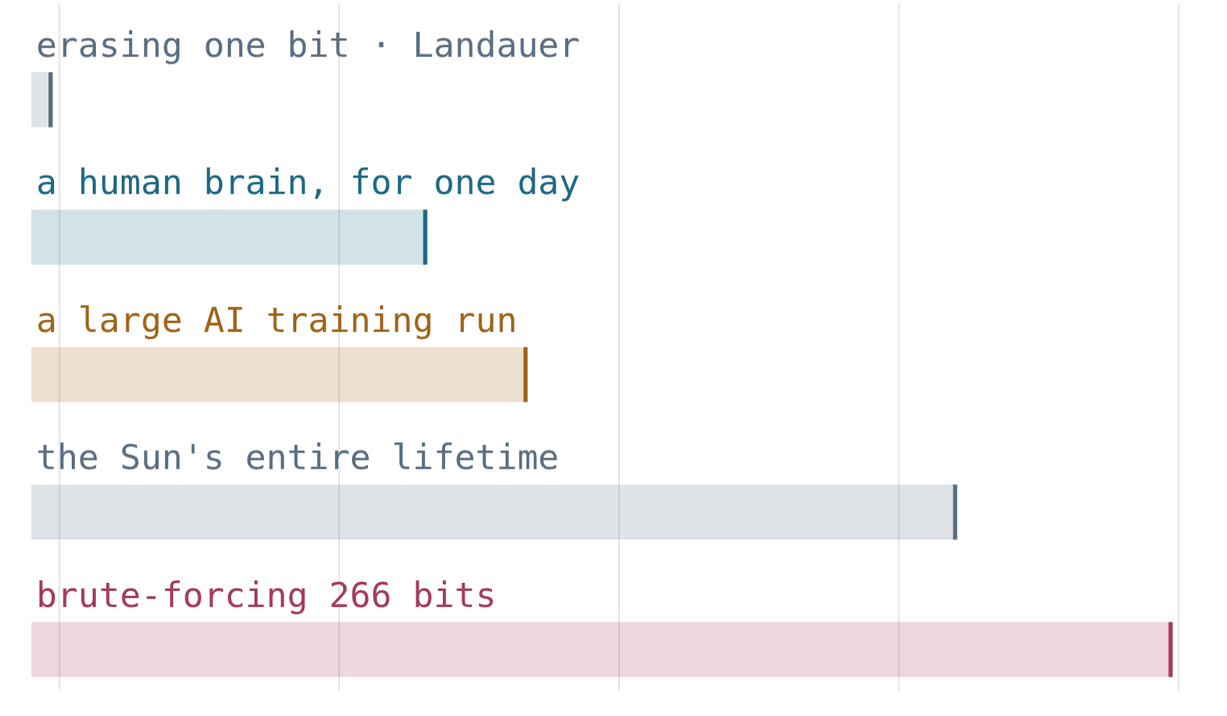
Composing a symphony versus hearing it is off. Writing a proof versus reading it. Designing a protein versus assaying it. Doing the homework versus grading it.

REMEMBER THIS ONE

Generation is expensive; verification is cheap. Every healthy division of labour — and every safe way to use an AI — is built on that gap.

PHYSICS

Computation is not abstract. Every step that forgets costs energy.



Landauer, 1961: erasing one bit at room temperature costs at least $kT \ln 2 \approx 3 \times 10^{-21}$ joules — because erasing is irreversible: two states become one, and the lost distinction leaves as heat. Information is physical.

A search moves on, and moving on is erasing. Just *counting* through our 266-bit space flips the lowest bit 2^{266} times, each flip an overwrite: $\approx 10^{59}$ J, about 10^{15} times everything the Sun will ever radiate. You cannot even count the states.

Meanwhile a human brain runs on 20 watts: a dim light bulb, doing what data centres cannot.

The bill is for any machine that forgets as it goes — every machine ever built; reversible computing lowers it in principle, at the price of time and of error correction, which forgets again. Time, space, energy: one budget. Any claim about intelligence that ignores it is a claim about magic.

REFRAME · 4 OF 5

Hardness is load-bearing.

WHAT IT FORBIDS

Brute force as a strategy. You cannot search your way to correctness, to a cure, or to a business plan.

WHAT IT OPENS

Every private message, every signature, every payment, every password you rely on today exists *because* some problems are hard. Difficulty is the raw material of security — and the reason abstraction, theory, and taste are worth more than compute.

If exhaustive search worked, there would be no cryptography, no privacy, and — since anything findable would be found — nothing left to discover.

METRICS

A language without a metric is decoration.

Notation lets you say it. Semantics fixes what it means. A metric tells you whether you are getting closer.

Running time, memory, energy, error rate, coverage, p-values, yield, latency, mortality, loss on a held-out set — every mature field runs on invented measures.

And every metric decays the moment it becomes a target.

> *When a measure becomes a target, it ceases to be a good measure.*
>
> GOODHART'S LAW · 1975

Teaching to the test. Optimising engagement. Publishing to the h-index. Training to the benchmark. In 1989 the CAST trial found drugs that suppressed the arrhythmia and raised the death rate. Same failure, five fields. A metric is a semantics for "better" — always approximate, and sometimes fatally so.

PART V

Everywhere

None of this is about computers. Computers are just where it was noticed first, because there the notation is forced to be exact.

BIOLOGY

Life was running a formal language long before we invented one.

Syntax: 3.2 billion base pairs, two bits each — about 800 megabytes. Smaller than a phone's memory; state space $4^{3,200,000,000}$.

Semantics: the genetic code maps 64 codons onto 20 amino acids and a stop — a redundant, many-to-one interpretation function, executed by the ribosome.

Self-reference: the genome encodes the machinery that reads the genome. Von Neumann described this architecture in 1948 — *five years before* Watson and Crick.

INTRACTABILITY · LEVINTHAL 1969

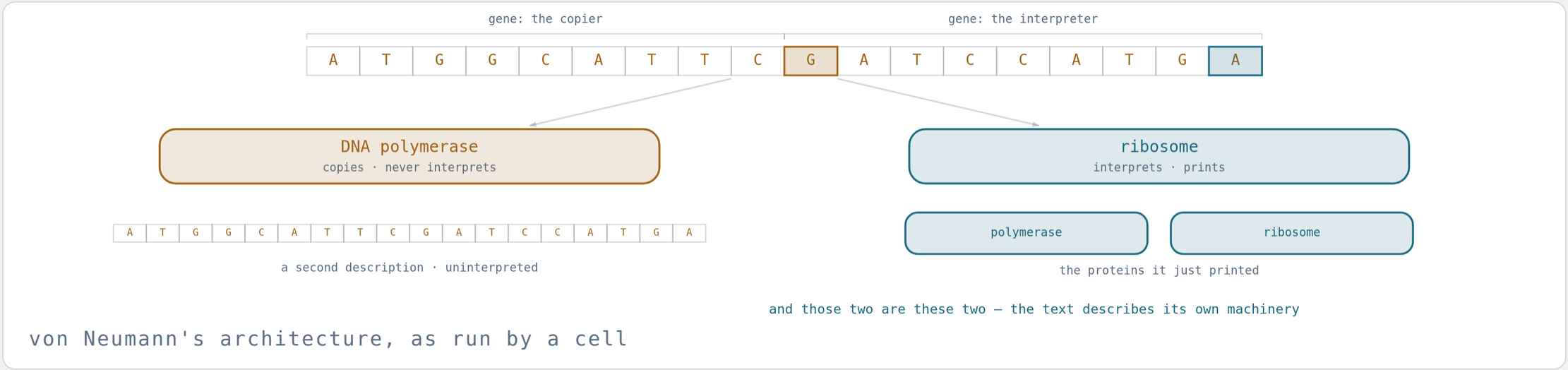
A modest protein has ~10^{300} possible shapes. Sampling them all would outlast the universe; real proteins fold in milliseconds. Nature does not search — it *funnels*.

AND WHERE AI ACTUALLY WON

AlphaFold did not solve folding by exhaustion either. It learned an approximation, and its value was settled by a metric the field had already invented and by experiments it could not fake.

BIOLOGY · SELF-REFERENCE

A copier that never interprets, and an interpreter that prints the copier and itself!



von Neumann's architecture, as run by a cell

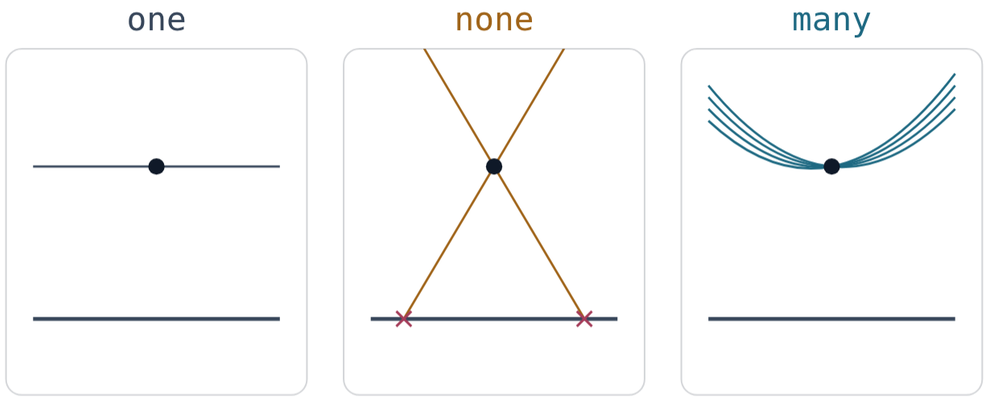
Copy. Polymerase walks the strand and duplicates it letter by letter, never asking what a letter means. Notation to notation.

Interpret. The ribosome walks the same strand and *executes* it — an interpreter in the sense a 3D printer is one: description in, object out. Notation to meaning.

Close it. Among the objects it prints are the polymerase and the ribosome — and one ribosome prints *any* protein, because which protein is data on the strand. That is U from Part III: universality, in chemistry.

MATHEMATICS · PHYSICS · CHEMISTRY

The postulate nobody could prove was a door, not a wall.



all three are consistent · the postulate was a choice

one line, one point beside it · how many parallels?

Euclid's fifth: through a point beside a line there is exactly *one* parallel. For two thousand years everybody failed to prove it.

In the 1820s Bolyai and Lobachevsky changed the postulate instead. Many parallels, or none — nothing breaks.

Unprovable because independent: a choice, not a fact. Gödel's shape, a century early — and the way out was a new language, not more effort.

AND THEN THE NOTATION KNEW FIRST

Riemann built curved-space geometry in 1854 for nothing in particular; Einstein could not have written relativity without it. Mendeleev left *holes* in his table and named elements nobody had seen. Dirac's equation had a solution nobody wanted; the positron turned up in 1932.

PSYCHOLOGY · SPACE AND TIME

Working memory holds about four things — and what counts as a thing is up to you.

F B I C I A N A S A U S B — 13 items · read once, look away, repeat

FBI CIA NASA USB — 4 items · the same 13 letters, in the same order

SPACE BOUND · MILLER 1956

Seven items, plus or minus two; later work says four. That number never grows. But an item is one *symbol* of a language you already own: a chess master shown a real position for five seconds puts nearly every piece back — shown a random board, barely better than a beginner.

Cowan 2001 · Chase & Simon 1973

TIME BOUND · KAHNEMAN 2011

System 1 answers from what is already stored: instant, cheap, *plausible* — and sometimes wrong. System 2 derives the answer: slow, expensive, checkable. Part IV's cost problem inside one head — a lookup against a proof.

System 1 / System 2

WAY OUT · PIAGET, HARNAD

Input fits the scheme you have? *Assimilate.* Does not fit? *Accommodate* — rebuild the scheme. A new property, or a new language: the definition this talk ends on, run by a child. And a symbol is worth only what it is *grounded* in.

accommodation · symbol grounding 1990

The four slots never grow. What grows is what one slot can mean — that is what growing up is, and what a discipline is, run on one person.

THE PATTERN

Every leap in every field was a new notation.

FIELD	NOTATION INVENTED	WHAT BECAME POSSIBLE
accounting	double-entry ledger · Pacioli 1494	the firm, the audit, capitalism
music	staff notation · Guido d'Arezzo c.1025	polyphony, composition at scale
painting	linear perspective · Alberti 1435	depth, and a space to compose in
physics	calculus · Newton & Leibniz 1670s	motion, orbits, engineering
chemistry	formulae & the periodic table · 1869	prediction of unknown elements
medicine	diagnostic criteria, trial protocols	evidence instead of authority
linguistics	generative grammar · Chomsky 1957	language as a formal object
computing	Turing machines · 1936	all of the above, mechanised

None of these was a discovery of new facts. Each was a new *language* — or a new *property* an old language could finally express. The facts followed.

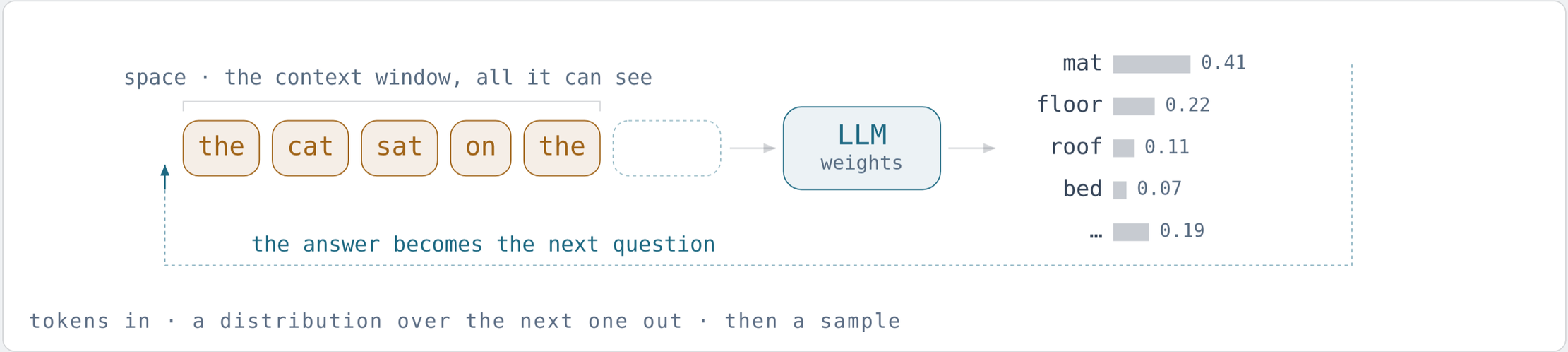
PART VI

Machines

Now the question everyone actually came with. Today's AI is remarkable — and it is subject to every single thing we have just established.

WHAT IT IS

A large language model is a machine trained on notation, asked for meaning.



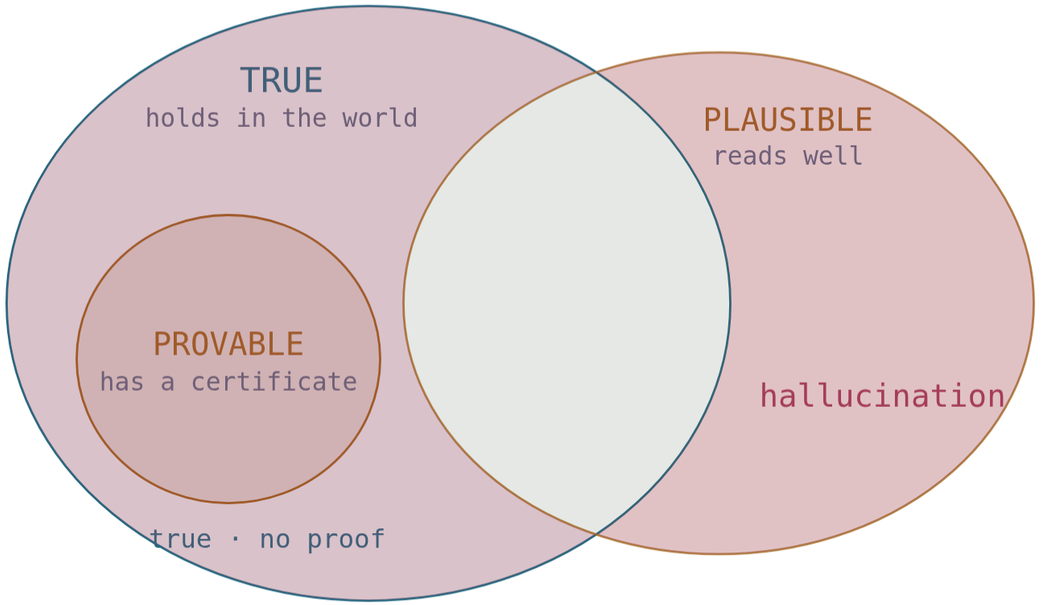
The name is exact. What a large language model — an LLM — is trained on is *language*, so the signal is syntactic: which symbol follows which. Meaning is never handed to it.

That this works as well as it does is the genuine surprise of the decade. It cost megawatts for weeks; you run on twenty watts. Neither figure changes what kind of object it is.

Billions of parameters, each many bits. Part I applies unchanged: that space cannot be inspected, so the behaviour cannot be enumerated. Only sampled.

THE OLD GAP, INDUSTRIALISED

A hallucination is a proof-shaped object that isn't true.



three tests · and they are not the same test

Fluency is syntax. Correctness is semantics. We built a machine of extraordinary fluency, so the gap between the two is something you now meet before breakfast.

Not a defect to be patched away: it is the proof/truth distinction at consumer scale.

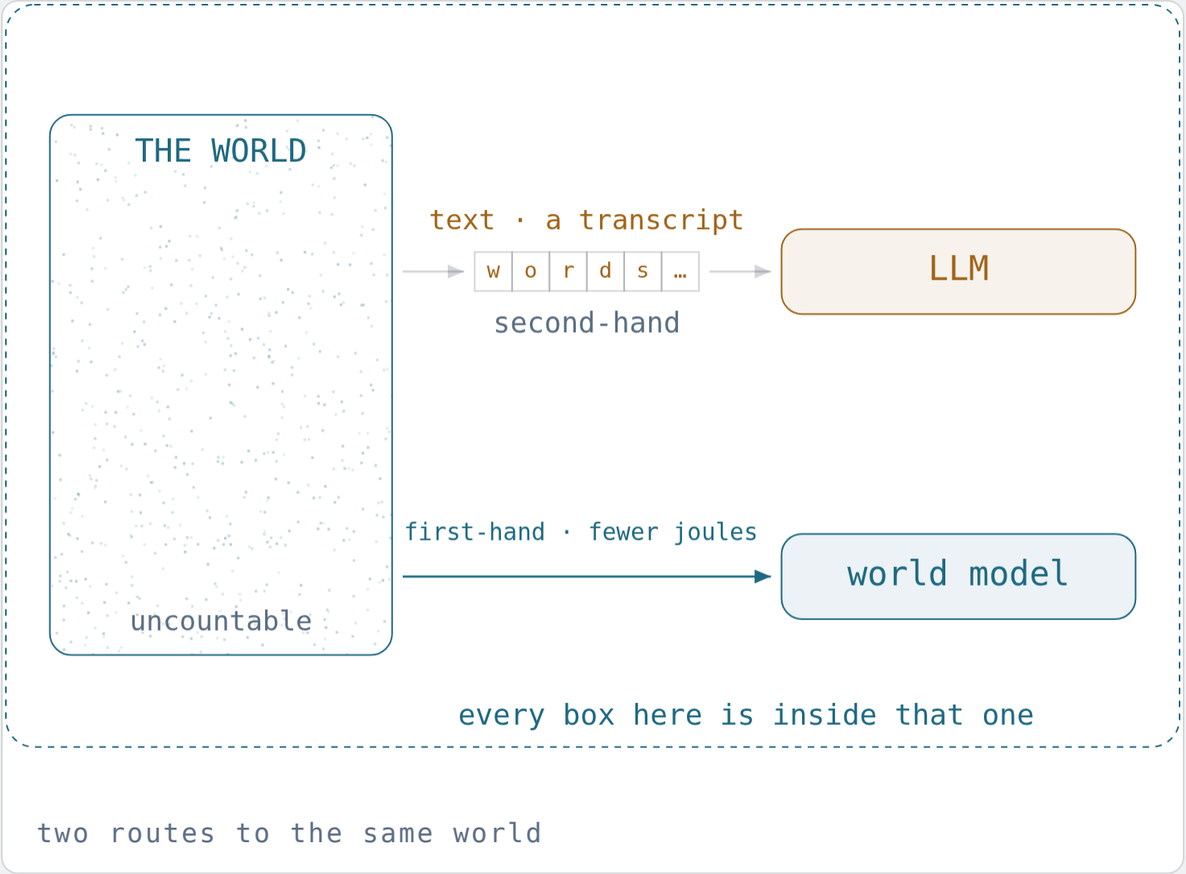
So the durable response is not "trust it more" or "trust it less" but check it against something with a semantics — a compiler, a test, an experiment, a source, a colleague.

GROUNDING AGAIN

Harnad, 1990: symbols defined only by other symbols never touch the world. Tools and feedback plug that hole partially, never completely.

WHERE IT IS GOING

World models aim at meaning itself — and inherit every limit anyway.



Not more text but a model of the *world* — state, dynamics, consequence. Predict what happens, not what is said: an attempt at real semantics.

And a bid for efficiency — fewer examples, fewer joules. On a fixed budget *efficiency is capability*, so world models may well outperform LLMs outright.

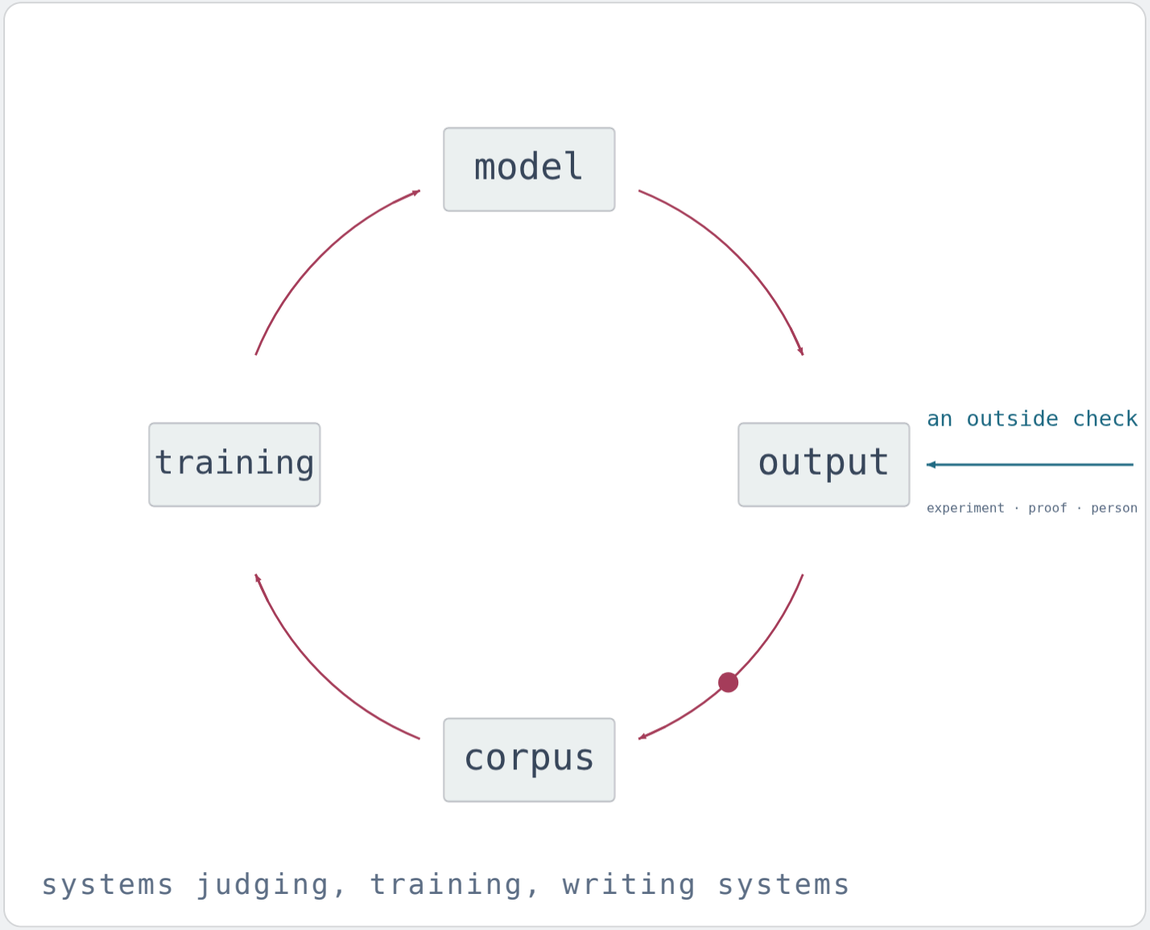
And still a finite notation, sitting inside the world it models. Parts II–IV apply unchanged.

WHAT IT INHERITS

Counting — models are countable, behaviours are not.
Rice — "is this model right?" is semantic. Undecidable.
Gödel — no self-certificate from inside.
Cost — the state space did not shrink. Only the map did.

SELF-REFERENCE · AGAIN

The loops are already closed.



— Models are trained on text that models wrote — with measurable degradation when the loop tightens (*model collapse*, 2024).

— Models grade models. The judge and the candidate share the same blind spots.

— Models write the code that trains models, and agents invoke themselves.

— "Is this system safe?" is a semantic property of a program. Rice says: no general decision procedure. Not hard — impossible.

— And by Gödel's second theorem, no system this expressive can certify itself. An explanation of its own output is more output.

WHAT ACTUALLY CHANGED

Generation got cheap. Verification did not.

BEFORE

Producing a draft, a proof sketch, a program, an image, a translation was expensive and therefore scarce. Scarcity did our filtering for us.

NOW

Production is nearly free and unbounded in volume. The filter has to be supplied deliberately — by specification, by measurement, by review.

Recall the asymmetry from Part IV: hard to find, easy to check.

That is a description of a healthy relationship with a machine.

Value migrates to the two ends AI does not occupy: deciding what should be true (specification) and establishing that it is (verification). Both are acts of meaning. Neither is automated by better generation — and the better generation gets, the more they are worth.

PART VII

Practice

A method that does not care which model is current, which company leads, or what happens next year.

THE QUESTION, REPLACED

Stop asking whether it is intelligent.

UNDECIDABLE · UNPRODUCTIVE

Is it intelligent? Does it truly understand? Is it conscious? Will it always behave? Each asks for a semantic verdict on a system, in a language with no semantics for the words used.

DECIDABLE · TODAY

What language did I state my requirement in? What are its semantics? What is my metric, and how will it be gamed? What is the budget? Who checks the result, from outside?

The right question is never about the machine's *essence*.

It is about your language, your semantics, your metric, your check.

This is why the method is future-proof: it never mentions a model, a vendor, or a year. The same five questions worked for the steam engine and the spreadsheet, and they will work for whatever arrives after transformers.

TAKE THIS WITH YOU

Six habits with no expiry date.

1. **Name the language.** State what you want in something with a semantics — a type, a schema, a unit, a test, an acceptance criterion. English negotiates; formality commits.
2. **Attach a metric, and name its failure.** If you cannot say what "better" means, you are wandering. Then write down how the metric will be gamed — it will be.
3. **Delegate generation. Own verification.** Accept work you could not have produced; never work you cannot check. That is the only shape delegation has ever had.
4. **Budget time, space, and energy.** Know how big your space is and how much of it you can afford to visit. Most doomed projects were doomed arithmetically on day one.
5. **Break every loop from outside.** Nothing certifies itself — not a model, not a company, not you. Import an independent check: an experiment, a proof, a person allowed to say no.
6. **Trust the unproven, then formalise it.** When answers get hard to check, what is missing is a language, not more effort. Take the truth you can see but not yet prove — and build the notation that proves it.

PRECEDENT

No theorem says your bridge will hold. You crossed one this morning.

Aviation, medicine, and civil engineering never had decidability either. Their questions are semantic, their state spaces astronomical, their proofs unavailable.

They became extraordinarily safe anyway — with layered approximations: standards, redundancy, staged testing, incident reporting, licensing, liability, and the freedom to say *not yet*.

None of it is proof. All of it is accumulated, measured approximation. It worked.

THEREFORE

"We cannot prove it safe" was never a reason for paralysis and never a licence for recklessness. It is the normal condition of every mature engineering discipline — and the reason those disciplines built institutions instead of theorems.

AI is early in exactly that process. Being early is a job opening, whatever you are studying.

BACK TO THE SHORT ANSWER

Every notation was invented by someone who saw a truth before it could be proved.

Fermat asserted it; Wiles proved it 358 years later. Riemann's hypothesis has been assumed true, and built upon, for 165 years. Mendeleev left blank cells for unseen elements. Darwin had no genetics.

In each case the notation came *after* the conviction — and the conviction was not deduced from anything.

Gödel makes this precise: no mechanical procedure yields the next axiom. Deciding what to hold true is not a step inside the system. It is a step to a new one.

THEREFORE, SYMMETRICALLY

This step is not automatable — not because machines are weak, but because there is no algorithm there to automate. Scale does not help; it was never a compute problem.

THE CHALLENGE FOR ALL OF US

Machines are already formidable at proving *within* a language. The open frontier — for every student here and for every AI ever built — is finding promising unproven truth, and then building the language or the property that captures it.

THE DEFINITION

So — what is intelligence?

WORKING DEFINITION

> *Intelligence is developing new formal languages — or at least new properties in existing ones — which requires finding and understanding promising unproven truth. New languages and properties let us ask new questions about that truth, and then answer them in proofs. Forever.*

ASKING THE RIGHT QUESTIONS

Choosing which unproven truth is worth formalising. Not deducible, not teachable by rote — this is what advanced, graduate-level study is *for*.

DEVELOPING THE SKILLS TO ANSWER

Proving, computing, measuring, building. Rigorous, cumulative, teachable — this is what an undergraduate programme gives you.

Not a threshold, a score, or a possession — an activity with a direction and no terminating condition. And notice where the difficulty sits: not in the *proving*, which machines do superbly, but in the finding, which nothing yet does reliably.

WHY DEPTH STILL MATTERS

This is exactly why you study a field in depth.

Not because AI is unavailable — it is extravagantly available, and it will get better every year of your career.

But finding and understanding promising unproven truth requires having lived inside a subject long enough to feel where it is thin, where it is wrong, and where it is about to give.

No summary, no search, and no generated answer transfers that. It is built, slowly, and it is yours.

AND IT NEED NOT BE COMPUTER SCIENCE

Every field has unproven truth in front of it — history, medicine, ecology, music theory, economics, law. The method in this talk is field-independent; the depth has to be specific.

Pick the field — or fields — whose unsolved problems still make you strangely comfortable.

REFRAME · 5 OF 5

Truth can only be approximated.
The approximating never stops.

WHAT WE LOST	WHAT WE HAVE INSTEAD
Completeness. Certainty. A final theory, a final language, a decision procedure for meaning, and any machine that could hand us all of it.	Work that cannot be finished, and therefore cannot be taken away: an unbounded frontier where every new notation makes yesterday's unreachable truths reachable, on every single day, forever.

The halting problem, applied to progress, gives the most hopeful result in science:

this computation does not terminate.

SOURCES · AND WHERE TO GO NEXT

Further reading.

THE THEOREMS

Cantor 1891 · Russell 1901 · Gödel 1931 · Tarski 1936 · Turing 1936 · Rice 1953 · Cook 1971 · Karp 1972 · Levin 1973

PHYSICS OF COMPUTING

Shannon 1948 · Landauer 1961 · Bremermann 1962 · von Neumann, *Theory of Self-Reproducing Automata* 1966

OTHER FIELDS

Frege 1892 · Wittgenstein 1921 / 1953 · Korzybski 1931 · Miller 1956 · Chomsky 1957 · Wigner 1960 · Levinthal 1969 · Chase & Simon 1973 · Goodhart 1975 · Harnad 1990 · Cowan 2001 · Kahneman 2011 · Jumper et al. 2021

READ FOR PLEASURE

Borges, *The Library of Babel* 1941 · Hofstadter, *Gödel, Escher, Bach* 1979 · Dijkstra, *EWD* notes · Chaitin on randomness

HANDS ON

Kirsch, *Elementary Computer Science: From Bits and Bytes to the Universality of Computing* — and the selfie system it is built on.

`github.com/cksystemsteaching/selfie`

KEYS

→ ← steps · ↑ ↓ slides · n notes · t clock · d theme

CLOSE

Notation is finite. Meaning is not. Mind the gap — then go widen it.

You are not competing with a machine at producing notation. You are the part of the loop that decides what it should *mean*, and whether it is *true*.

ASIDE · FOR THE COMPUTER SCIENTISTS IN THE ROOM

If you want to hold self-reference in your hands: *selfie* is a 12KLOC system in which a compiler compiles itself and an emulator executes itself — plus *monster* and *rotor*, which translate real machine code into logical formulae, so you can watch syntax become semantics and then run straight into NP-completeness on your own laptop. `github.com/cksystemsteaching/selfie`

Two slides left. One is a joke. One is the point.

PENULTIMATE

Whatever intelligence is, humour is the only way.

For most of you, this hour has probably confirmed your conviction never to study computer science.

Or the exact opposite. And who knows — some of you may now be considering a switch *into* computer science. I have seen it happen. (With apologies to my colleagues in the other fields.)

OUTCOME A

Confirmed conviction.

OUTCOME B

Informed confusion.

Regardless of which one you leave with: this is about becoming who you are and no one else.

> *Denn es ist zuletzt doch nur der Geist, der jede Technik lebendig macht.*
>
> JOHANN WOLFGANG VON GOETHE

For in the end it is only the Geist that brings any technique to life.

Geist — spirit, mind, meaning. For the last hour we have been calling it truth. And it is, finally, you and me.

Technik — technique, technology, the formal machinery. Necessary. Never sufficient. Never alive on its own.